CE152: Object-Oriented Programming

2D Graphics - Part 1

Dr. Sebastian Halder

CSEE

2023

2D Graphics

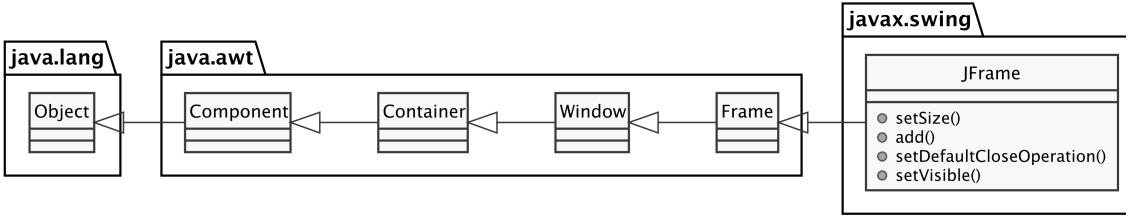
Java 2D library

- Java has an extensive graphics library
  - We can draw lines or geometrical shapes
  - Fill those shapes with colours
  - Draw text
  - Draw images and filter them
  - Apply transformations to images and shapes
  - Perform hit detection
  - Write the images to JPEG or PNG

Getting started with 2D graphics

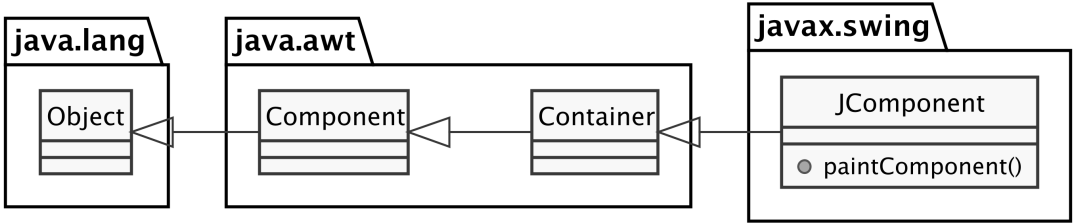
- We need a window to draw on
  - In Java this is called a frame and the class is *JFrame*
  - We can choose the title and size of this frame and add components to it (see below)
- We need a drawing to put into this window
  - In Java this is called a component and the class is *JComponent*
  - This component will have a method called *paintComponent* that is automatically called when the frame becomes visible
  - In this method we will create Graphics2D Objects that are the actual graphical shapes (such as lines or circles)

Java JFrame UML



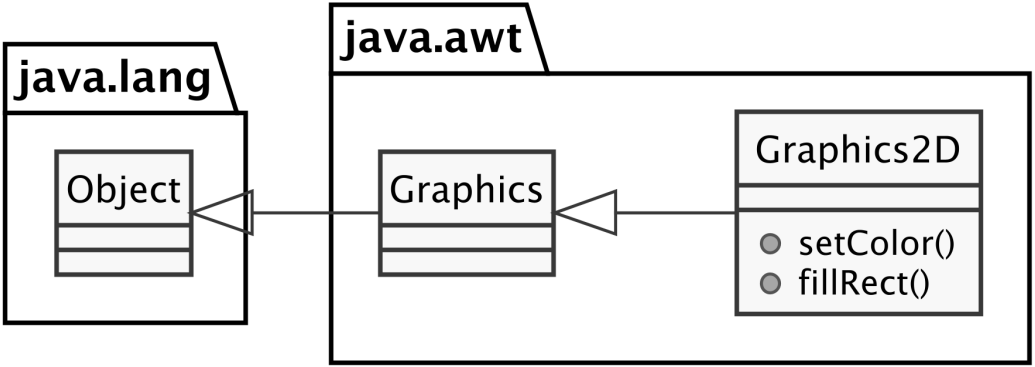
- By the way, AWT is the abstract window toolkit and an older version of Swing that is used as a basis
- We would call lang, awt and swing *namespaces*
- We can make our own namespaces using the keyword *package* before the class definition
- JFrame has many more methods then listed here... please have a look using IntelliJ or in the documentation

Java JComponent UML



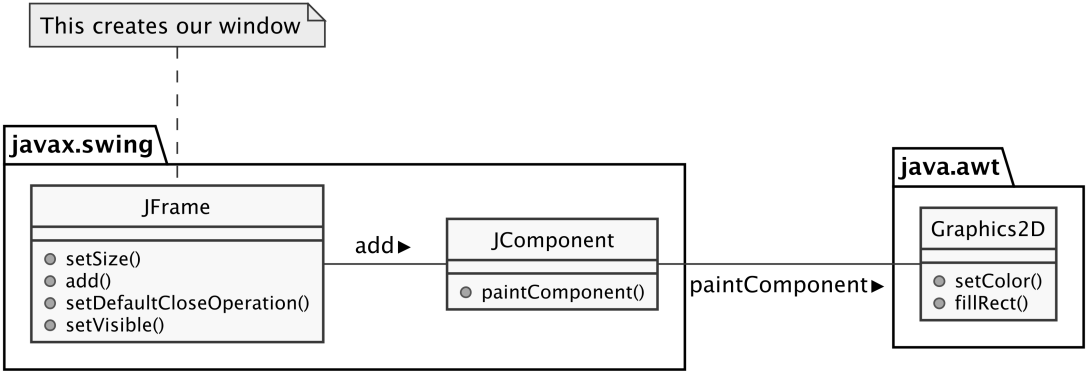
- Again JComponent has many more methods then listed here. . . please have a look using IntelliJ or in the documentation

Java Graphics2D UML



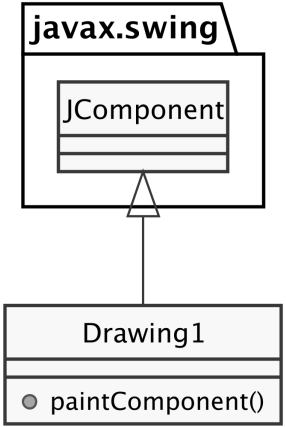
- Also Graphics2D has many more methods then listed here. . . please have a look using IntelliJ or in the documentation

Simple 2D Graphics summmary UML



- The connections here do not signal inheritance!

Our Drawing1 in UML



Our Drawing1 with Graphics2D in Java

```java
import javax.swing.*;
import java.awt.*;

public class Drawing1 extends JComponent {
    @Override
    public void paintComponent(Graphics graphics) {
        // Whats what we are doing below called?
        Graphics2D g = (Graphics2D) graphics;
        // add drawing code below
        g.setColor(Color.GREEN);
        g.fillRect(100, 100, 200, 100);
    }
}
```

JFrame we instantiate in main

```java
import javax.swing.*;

public class Main2D {
    public static void main(String[] args){
        // What does the parameter do?
        JFrame f = new JFrame("Drawing 1");
        // What do these methods do?
        f.setSize(600, 600);
        f.add(new Drawing1());
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Some more drawing examples

```
drawLine (int x1, int y1, int x2, int y2)
drawPolyline (int[] xPoints, int[] yPoints, int nPoints)
// if applicable methods exist for drawing the outline
// and the filled version
drawRect(int x, int y, int width, int height)
fillRect(int x, int y, int width, int height)
drawOval(int x, int y, int width, int height)
fillOval(int x, int y, int width, int height)
drawString (String str, int x, int y)
draw (Shape s)
```

Even more drawing examples

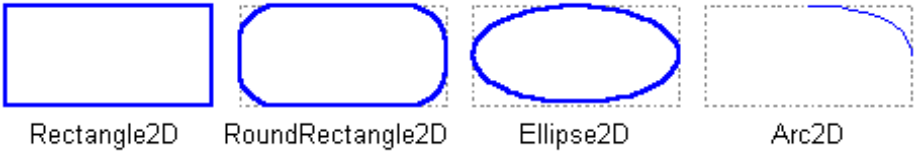
```
// set the current colour
setColor(Color c)
// set the current font
setFont(Font font)
// translate, scale or rotate graphics content
translate(int x, int y)
scale (double sx, double sy)
rotate (double theta)
// set the background color
setBackgroundColor(Color c)
// clear rectangle by filling it with background color
clearRect(int x, int y, int width, int height)
```

The coordinates explained

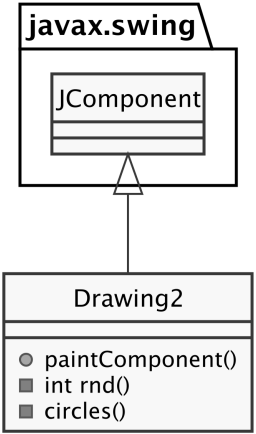
- When we draw a shape we supply the coordinates
- These coordinates originate *in the top left corner* of the frame
- This is called *user space coordinate*
- For rendering the objects are transformed to *device space coordinates*

Bounding box

- We call shapes such as rectangles and ovals *geometric primitives*
- All shapes are defined by their bounding box
- Circles are special ovals, squares are special rectangles
- Note the coordinates refer to the top-left not the centre!
- See tutorial



Drawing more things at once! (UML)



- We want to create 50 random circles with random colors
- So we add a method that generates 50 circles of random size and color whenever paintComponent() is called
- The circles() method uses the rnd() method to generate the circles

The Java code for drawing more things at once

```
import javax.swing.*;
import java.awt.*;

public class Drawing2 extends JComponent {
    public void paintComponent(Graphics g) {
        circles((Graphics2D) g);
    }

    private int rnd(int n) {
        return (int) (Math.random() * n);
    }

  // continued on next slide
```

The code continued

```
  // continued from previous slide
  // where do we change the number of circles?
  // in which line should we avoid using a constant?
  // Remember: g is a reference
  private void circles(Graphics2D g) {
    for (int i = 0; i < 50; i++) {
        g.setColor(new Color(rnd(256),
          rnd(256), rnd(256)));
        int r = 5 + rnd(60);
        g.fillOval(rnd(600), rnd(600), r, r);
    }
  }
}
```

Live coding

- Create JFrame with JComponent and add one geometric primitive
- Draw multiple primitives

Plotting a mathematical function

- Task: plot

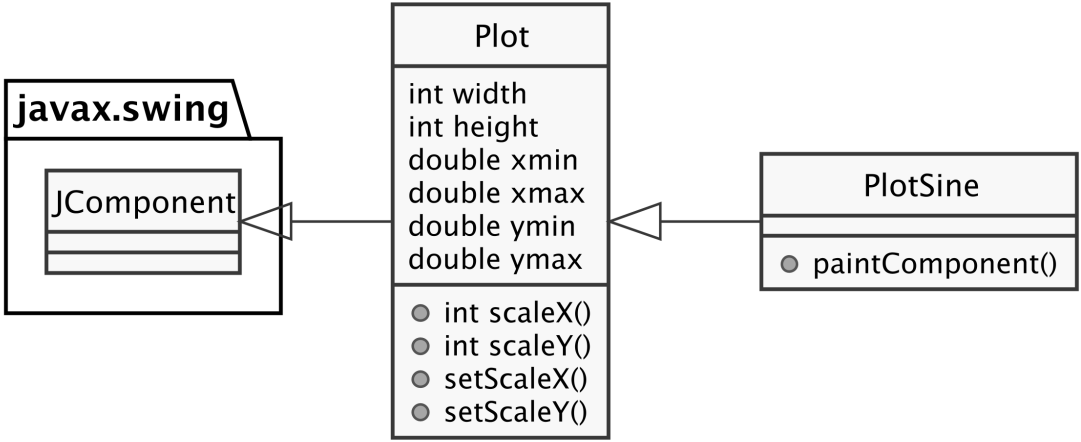
$$y = sin(4x) + sin(20x)$$

  for all

$$x \in [0...\pi]$$

- Approach:
  - Calculate points and store them in two arrays (one for x and one for y)
  - Draw them using *drawPolyline*
- The class will contain some methods to scale method coordinates to the current screen coordinates
- We will re-use this class in other drawing programs

Plot method UML



Plot method Java

```
import javax.swing.*;import java.awt.*;
// transformation of user coordinates to screen coordinates
public class Plot extends JComponent {
    // dimensions of plotting area with default values
    int width = 600, height = 600;
    // dimensions of user-space coordinates with
    // default values
    double xmin=0,  xmax=1, ymin=0, ymax=1;
    // transformation of coordinates
    public  int scaleX(double x) {
        return (int) (width * (x - xmin) / (xmax - xmin));
    }
    public  int scaleY(double y) {
        return (int) (height * (ymin - y)/(ymax - ymin)
              +height);
    } // continued on next slide
```

Continued

```
    public  void setScaleX(double min, double max) {
        xmin = min;   xmax = max;   }
    public  void setScaleY(double min, double max) {
        ymin = min;   ymax = max; }
  }
```

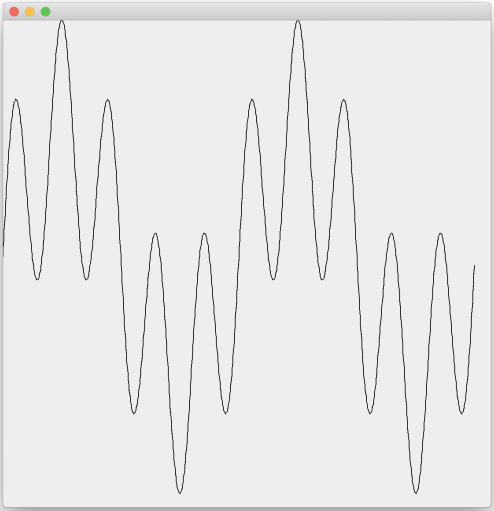
PlotSine in Java

```java
import java.awt.*;

public class PlotSine extends Plot {
    public void paintComponent(Graphics g) {
        // What effect can we observe when we set
        // the N to a low value?
        final int N = 1000;
        int[] x = new int[N];
        int[] y = new int[N];
        for (int i = 0; i < N; i++) {
          double xi = Math.PI * i / N;
          x[i] = this.scaleX(xi);
          y[i] = this.scaleY(Math.sin(4*xi) + Math.sin(20*xi));
        }
        g.drawPolyline(x,y,N);
    }
```

Using the class

```
import javax.swing.*;

public class Main2D_2 {
    public static void main(String[] args) {
        PlotSine myPlot = new PlotSine();
        myPlot.setScaleX(0.0, Math.PI);
        myPlot.setScaleY(-2.0, 2.0);
        JFrame f = new JFrame();
        // make frame a bit larger than plotting area
        f.setSize(myPlot.width + 20, myPlot.height + 40);
        f.add(myPlot);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

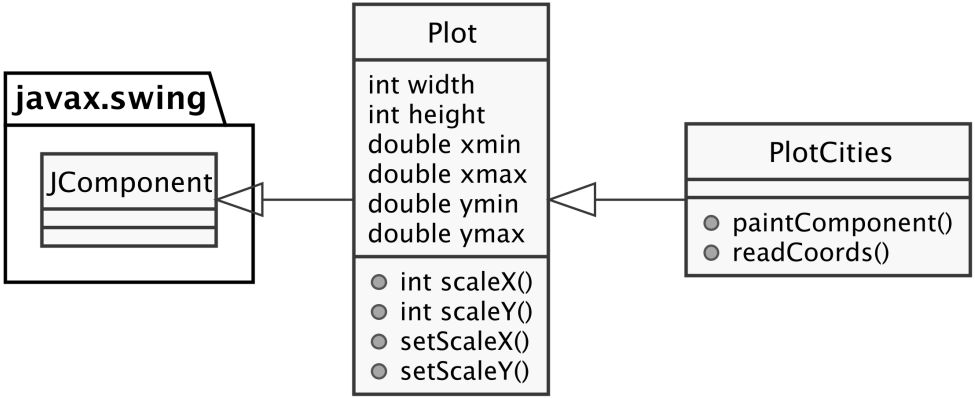
The plot



Data Visualisation

- usa.txt contains the coordinates of over 13k US cities
  - The first line of the file contains the number lines and the minimal and maximal x- and y-coordinates
  - Each subsequent line contains the coordinates of one city
- Task: visualise the data from this file
- Approach:
  - We will read the coordinates from the file into an array called coords and transform them using scaleX and scaleY

Plot cities UML



Plot cities Java

```
import java.awt.*;
import java.io.File;
import java.util.Scanner;

public class PlotCities extends Plot {
    int[][] coords;

    public void paintComponent(Graphics g) {
        for (int i = 0; i < coords.length; i++)
            g.fillOval(coords[i][0], coords[i][1], 3, 3);
    }

    public void readCoords() throws Exception {
        Scanner input = new Scanner(new File("usa.txt"));
        int n = input.nextInt();
        //continued on next slide
```

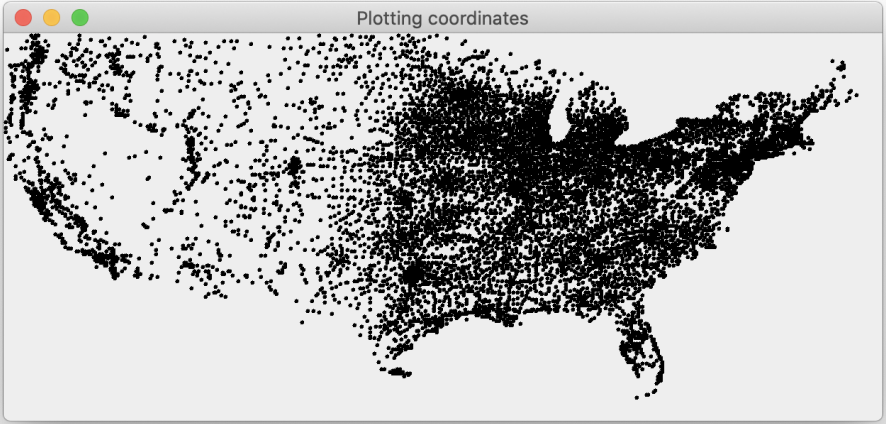
Continued

```
        coords = new int[n][2];
        // set up plot transformation
        double xmin = input.nextDouble();
        double ymin = input.nextDouble();
        double xmax = input.nextDouble();
        double ymax = input.nextDouble();
        this.height = (int) (this.width * (ymax - ymin)
            / (xmax - xmin));
        this.setScaleX(xmin, xmax);
        this.setScaleY(ymin, ymax);
        // fill coordinate array
        for (int i = 0; i < n; i++) {
            coords[i][0] = this.scaleX(input.nextDouble());
            coords[i][1] = this.scaleY(input.nextDouble());
        }
    }
```

Running PlotCities

```java
import javax.swing.*;

public class Main2D_3 {
    public static void main(String[] args) throws Exception {
        PlotCities myCities = new PlotCities();
        myCities.readCoords();
        JFrame f = new JFrame("Plotting coordinates");
        // make frame a bit larger than plotting area
        f.setSize(myCities.width + 20, myCities.height + 40);
        f.add(myCities);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

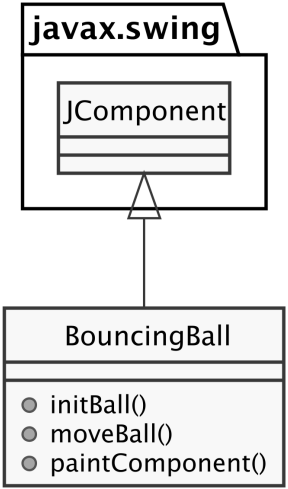
The result



A simple animation

- "Bouncing balls" is a well known simple animation
- We will implement it using a variable for ball position and velocity and methods for `initBall` and `moveBall`
- In `main()` we will use an infinite loop to keep the ball moving
  - In this loop `moveBall()` will be invoked to move the ball to a new position
  - Then the graphics will be repainted
  - The animation will pause for a certain number of ms

BouncingBall UML



BouncingBall Java (incomplete)

```
public void initBall(){
    ballX = BOX_WIDTH/2;
    ballY = BOX_HEIGHT/2;
    ballVX = (0.5 + Math.random()/2) * MAX_BALL_SPEED;
    ballVY = (0.5 + Math.random()/2) * MAX_BALL_SPEED;
}
public void moveBall() {
    ballX += ballVX;
    if (ballX < BALL_RADIUS || ballX
    + BALL_RADIUS > BOX_WIDTH)
        ballVX *= -1;
    ballY += ballVY;
    if (ballY < BALL_RADIUS || ballY
    + BALL_RADIUS > BOX_HEIGHT)
        ballVY *= -1;
}
```

Continued

```
public void paintComponent(Graphics g) {
    g.setColor(BOX_COLOR);
    g.drawRect(BOX_MARGIN, BOX_MARGIN, BOX_WIDTH,
      BOX_HEIGHT);
    g.setColor(BALL_COLOR);
    g.fillOval(BOX_MARGIN
      + (int)ballX - BALL_RADIUS,
      BOX_MARGIN + (int)ballY - BALL_RADIUS,
      2 * BALL_RADIUS, 2 * BALL_RADIUS);
}
```

The while loop in main()

```
animation.initBall();
while (true) {
    animation.moveBall();
    f.repaint();
    Thread.sleep(animation.REPAINT_INTERVAL);
}
```

Demonstration

- Run bouncing ball and plot cities

Summary

Java Library and Packages

- Java comes with many useful libraries
  - This is often called the API: *advanced programmer interface*
  - see https://docs.oracle.com/en/java/javase/index.html
- As we have seen the library classes are organised in Packages
  - `java.lang`, `java.util`, `java.net`, `java.util.regex`, `javax.swing`...
- Everything in `java.lang` is automatically known to the compiler
  - e.g. `String` would otherwise be `java.lang.String`
- We can use the keyword `package` to organise our own code in this way (for example to import the `Plot` class)

Import statements

- The `import` statements make classes accessible without having to write the full names
- Must be placed before any class declarations
- With the star symbol we can import all classes from one package
  - If this leads to a name clash we have to import individual classes instead
- We can import our packages to other packages in the same way
  - E.g. we can import MyMath from *video_arrays2*

The API is your friend!

- Make a bookmark to the Java documentation:
  - https://docs.oracle.com/en/java/javase/13/index.html
  - You can also find the documentation for other releases: https://docs.oracle.com/en/java/javase/index.html
- There is also an excellent tutorial:
  - https://docs.oracle.com/javase/tutorial/tutorialLearningPaths.html
- Try to use the library methods as much as you can!
  - This will make it easier for you to build and maintain code because you do not need to reinvent the wheel!
  - It also makes it easier for fellow programmers to read your code because they will know the same methods

Make sure you know the basics

- Today we saw a lot of different syntax for the first time
- The speed and complexity of the Java syntax we use will increase from now on
- If you are still struggling with the basics please go to the support class, the labs and study for at least four hours per week by yourself or with friends

Thank you and see you in the next video!